

个人实验报告（实验三）

桌面物体自动分类整理——真机执行（RoboMaster EP）

班级2024 届中外 2 班姓名马晨曦学号24020036047

小组角色任务控制与系统集成·真机执行主责日期2026-09-24

1 实验的基本要求

本实验为机器人集成小组项目 I 《桌面物体自动分类整理》，要求在实验一（视觉检测模型）与实验二（机械臂定点抓取）的基础上，做出一个“看得见—判得准—抓得起—放得对”的全自动整理系统：相机看桌面，识别出网格里摆放的多类物体，机器人依次把每个物体按类别放进对应料盒，直到桌面清空。

实验要求分为**阶段一仿真开发与阶段二真机验证**两段，且**阶段二的准入条件 = 阶段一仿真验收通过**（顺序不可颠倒）。本报告只写**阶段二真机部分**；阶段一仿真的验收结论只在下一小节照录一份，作为准入条件的凭据，细节归属仿真线负责人。

本版范围声明：阶段一仿真已于 2026-09-24 验收通过（见下），本报告据此成立。阶段二的**现场实测数据**——`config_real/` 的坐标与位姿表、现场运行产生的 `real_run.json` 与 `records.jsonl`、分类结果记录表——**须于现场作业后补录，本版不含，亦不预填任何估计值**；未标定时真机入口拒绝启动 (§3.4)，因此不存在“拿编出来的坐标跑一遍”的可能。

阶段一仿真的验收结果（阶段二的准入条件）

仿真线（mechArm + Gazebo, ROS 2）的**记分轮**已于 2026-09-24 跑通，

格号	类别	目标料盒	落料槽位	结果
c1	cup	bin_cup	0	成功
c2	mouse	bin_mouse	0	成功
c3	cup	bin_cup	1	成功
c4	cup	bin_cup	2（第 2 层）	成功
c5	mouse	bin_mouse	1（叠层）	成功
c6	cup	bin_cup	3	失败（放置偏差 52/87/17 mm）

结果文件 `exp3_logs/run_20260924_112831/result.json` 给出 `verdict=PASS`：场上物体 6 个，正确放入对应料盒 5 个（及格线 5），**规划失败 0 次**。

异常轮（`run_20260924_114058`）单独跑：c3 一格故意空置，`verdict=TRIAL`。空网格被跳过的证据不是某一行写了什么，而是 `records.jsonl` 里**没有 c3 这一行**、`result.json` 的 `reasons` 里也没有 c3 的键；其余 5 个全部放置成功。这一轮必然是 `TRIAL` 而非 `PASS`——判定规则只对**完整**的验收轮下结论，所以异常轮必须与记分轮分开跑。

口径说明（必须分开陈述）。本次仿真跑在没有 GPU 的云容器里，Gazebo 走软件渲染，跑到第 3~4 轮会出现**画面停滞**：帧戳照常推进而画面内容一字不变，依赖图像的 HSV 检测因此失效。为把故障隔离在渲染这一段、继续验证下游链路，记分轮改用**真值投影**生成检测框：物块世界坐标经同一份已标定的相机参数投影为像素，再走完全相同的“像素 → 桌面 → 格子”映射，相机标定与映射链路未作任何改动。因此这一轮 **能证明**的是相机标定、投影与网格映射正确，以及规划—取放—落料—回航这条执行链可行；**不能**声称 HSV 检测算法在整轮任务中稳定有效——渲染健康的第 1 轮 HSV 实测 6/6 正确映射，这份证据单独保留。

遗留：叠第 2 层时物块之间发生互穿，LCP 求解器迭代数冲到 122847 并把已放好的物块弹飞；第 2 层落差 $dz=0.036\text{m}$ 仅比杯高 0.035m 多 1mm ，加大落差或让层间 xy 错开是下一步要改的地方。c6 是记分轮唯一失败项。

真机任务（docx § 五）

- 在**真实作业区**（经确认在实验室地面上进行）标出取物网格与不少于 2 个分类料盒，摆放 2 类共 6 个实物；
- 使用**前序实验训练的检测模型**识别物体类别与所在网格；
- 使用**真机抓取程序**，把不同类别放入对应料盒。

真机验收标准（docx § 六）

- 6 个物体中至少 5 个被正确分类整理；
- 正常任务过程中不得进行人工类别判断和动作触发；
- 正确处理至少 2 类异常（空网格、未识别物体应被跳过；抓取失败或目标不可达时应返回安全位置或停止）；
- 提供一个入口启动图像、识别、机械臂和任务控制；
- 全程保存识别、抓取和任务状态日志。

真机平台的确定

docx § 实施原则写的“真实相机、Jetson 和 mechArm”是 mechArm 时代的条款。本组真机平台 **沿用实验二已完成 C5 验收的 RoboMaster EP 工程形态**（教师已于实验二阶段确认换平台，本实验沿用同一口径）：

- EP 为**工程形态**：全向底盘 + 二轴机械臂 + 夹爪，由**大疆官方 Python SDK** 直连控制，**真机侧不使用 ROS 2**，因此也不需要 Jetson；

- 相机为 **USB 接口摄像头**（与实验一同一款），固定于工作区上方；
- 仿真线仍为 mechArm + Gazebo (ROS2)，由组员甲负责。

注：本人承担**任务控制（状态机） / 接口契约 / 日志与验收口径**这条主线，并任**真机执行主责**——EP 侧标定、执行后端（EPPickExecutor）、真机入口（run_real.py）与现场运行录制。视觉模型来自实验一公开仓库，相机识别与仿真场景分别由两位组员完成。**本报告围绕真机执行这条线展开。**

2 个人分工

本人在真机阶段负责以下五件事：

1. **EP 执行后端 EPPickExecutor**：实现执行契约的**六段动作**（下探/夹取/抬离/搬运/松放/退回），每一段都落到实验二已验证过的 EP 原语上，并在失败时按段号映射为动作级原因；
2. **EP 侧标定与位姿表 config_real/**：底盘里程计位姿（6 网格 + 2 料盒 + 归位点）、臂的两档高度、夹爪档位，全部由现场实测回填；
3. **真机入口 run_real.py**：把“相机 → 检测 → 任务控制 → EP”串成一条命令，含启动前的**标定闸门**与异常收尾；
4. **未标定拒跑闸门**：把“点位到底标没标过”从一句口头约定变成**可执行的启动前检查**，避免拿没人量过的坐标去驱动真机械臂；
5. **真机运行与录制**：现场标定、单周期人盯跑通、全自动一整轮录制，以及验收日志与结果记录。

与本人的仿真线工作的关系：任务控制核心（sort_core）、接口契约（PickPlace）、原因枚举与日志 schema 是我在阶段一建立的，两条线**共用同一份**；真机阶段我做的是把**执行侧换成 EP**，核心一行未改（见 §3.1）。

组内分工概览（真机阶段）：**本人** = 真机执行主责（EP 标定 / 执行后端 / 入口 / 运行录制）+ 任务控制与契约；**组员甲** = 仿真场景搭建与联调、仿真演示视频、仿真与真机的尺寸对齐；**组员乙** = 识别链路（真模型）、真机相机标定、分类结果记录表。

3 项目实现的原理

3.1 双线架构：一套任务核心，两个注入缝隙

真机不是另写一套系统。任务控制核心 TaskController 只依赖两个函数缝隙 scan()（报一帧桌上检测）与 pick_place(cell_id, cls)（把指定格子的物体取走放好），自己不 import 任何 ROS 类型。真机线要做的只是**换掉这两个缝隙的实现**（图 1）：

- 缝隙 1：由“订阅 Detection2DArray”改为直接从 USB 相机取帧并调检测模型；
- 缝隙 2：由“发 PickPlace Action”改为直接调用 EPPickExecutor。

这条“决策与通信解耦”的设计是阶段一定的，真机阶段兑现了它的价值：契约、原因枚举、日志 schema 一行未改，换平台只换了接线。

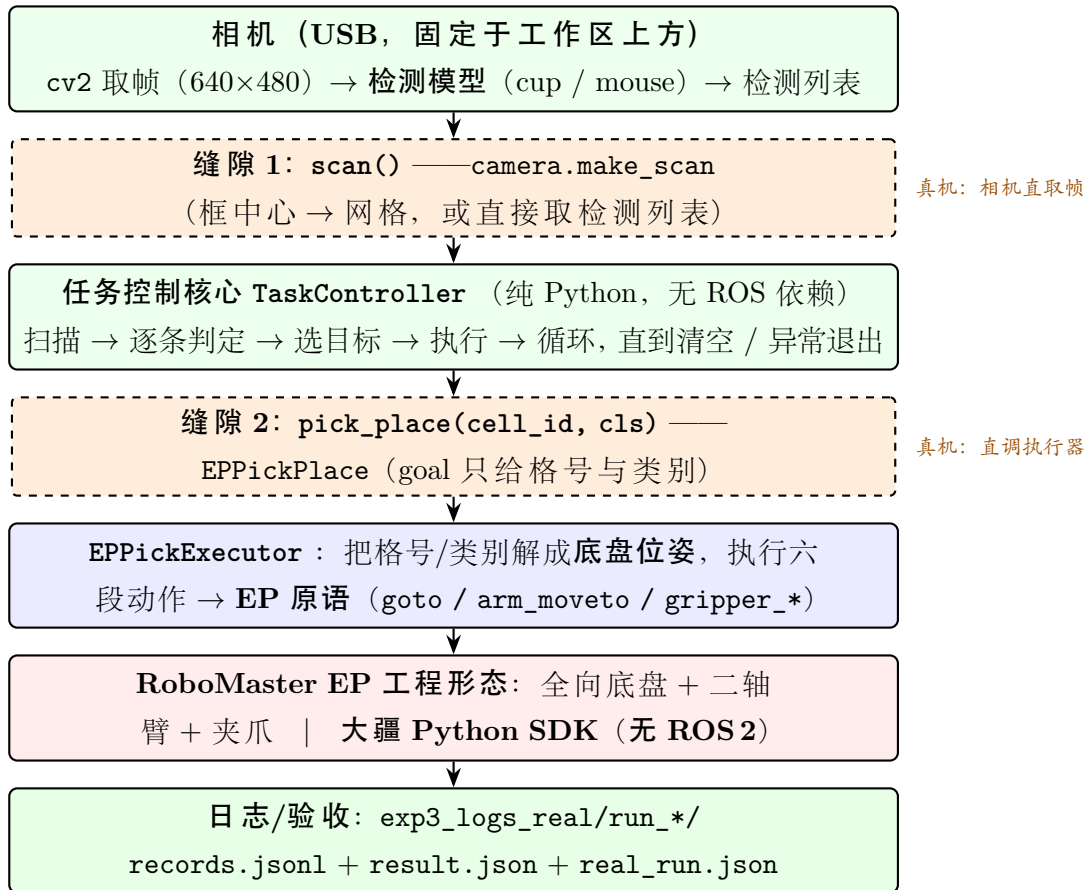


图 1: 真机线主链与两个注入缝隙（橙色虚线）。缝两侧口径固定，故任务核心与执行侧可分别开发、失败可分别定位；两处缝隙正是真机相对仿真唯一改变的地方。

3.2 一次取放拆成六段：把动作序列固定，把“在哪儿”参数化

“取一个物体并放好”是一个持续数秒的长任务，服务端把它切成固定的六个阶段，每进入一段先发进度，再交给执行者 PickExecutor。这个抽象是相对实验二最重要的一步泛化：实验二的直驱程序是“一个固定取物点 → 固定放置点”的循环，而本实验有 6 格 × 2 料盒；若直接套用就会退化成“每格写死一个周期”。真正会变的只是“目标和料盒在哪儿”，动作序列本身完全相同——于是把序列固定成六个方法、把位置压进参数（表 1）。

这张表背后有一条我给自己定的规则，值得单独说：只报这一段的代码真能做出来的判断，不替谁宣布一个没人做过的判定。真机上六段全都只能靠异常失败（只有抬离段和松放段会返回“否”，其余段没有可判的物理结果），所以这张表实际上是“按段解释那次异常”。两个具体后果：

段	语义	EP 原语	该段失败时的原因
1	下探	goto(格位姿) → arm_moveto(抓取低点)	unreachable_target
2	夹取	gripper_close()	exec_error
3	抬离	arm_moveto(抬升高点) + 判 held	grasp_failed
4	搬运	goto(料盒位姿)	unreachable_target
5	松放	arm_moveto(放置低点) → gripper_open() + 判 placed	dropped_or_missed_place
6	退回	arm_moveto(抬升高点) → goto(归位点)	exec_error

表 1: 六段动作与 EP 原语的对应，以及每段失败时映射出的动作级原因 (real/ep_backend.py 的 STAGE_REASON 为唯一权威)。

- **夹取段报 exec_error 而不是 grasp_failed**: 这一段的代码只是合一下夹爪，从没判过“夹住没有”（判定在下一段抬离）；而 grasp_failed 在原因枚举里的定义恰恰是“抬离后判定没夹起”。填它等于 **凭空宣布一个没做过的判定**——现场查日志的人会以为系统真的判过。
- **退回段报 exec_error 而不是 unreachable_target**: 仿真线这里用的是“不可达”，因为那个结论来自规划器；EP 没有规划层，没有任何东西产出过“不可达”。报驱动硬错才是如实描述。

这也解释了为什么真机线与仿真线在**退回段**上的原因刻意保持不同：不是没对齐，而是两个平台上“发生了什么”本来就不是同一件事。

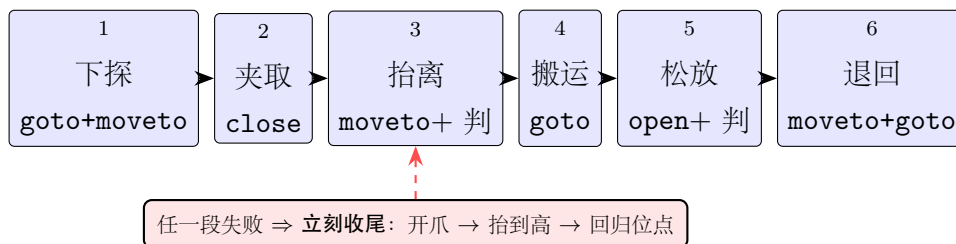


图 2: 六段动作链。与仿真线共用同一套段号；差别仅在原语是 EP SDK 而非 ROS 规划器。失败段号 → 原因 → 收尾这条路径见 §3.5。

3.3 EP 的位姿按编号查，不与桌面坐标换算（真机侧最关键的一处设计）

仿真线里，“格号 → 坐标”用的是**桌面坐标系**——相机标定出来的那一套。真机上我没有沿用这套，而是让每个格号/料盒号去位姿表里查一个**底盘里程计位姿**：

- **为什么不换算**：EP 底盘的位姿是里程计读数，以上一次上电的位置为原点；相机标定出的桌面系以桌面某一点为原点。两者之间没有已知的对应关系（相机装在哪、盘停在哪都不固定），而建立这个对应关系本身没有必要——接口只给格号与类别。
- **于是**：两套坐标各管各的，永远不用互相换算；换格子、换料盒、挪相机都只改各自那份配置，**不动代码、不动接口**。“换格不改接口”这句话在真机侧的收益就落在这里。
- **代价（如实记录）**：底盘是开环定位，里程计会漂移，且**每次上电原点重置**。所以每次换场地或重新上电，位姿表必须重标——这一点被写进了启动前检查 (§3.4)。

位姿表 `config_real/ep_waypoints.json` 的内容即为此：6 个网格位姿 + 2 个料盒位姿 + 1 个归位点 (goto 的目标)，臂的两档高度 (`grab_low_mm` 抓取低点 / `lift_high_mm` 抬升高点)，以及夹爪的开合档位与动作等待时间。

3.4 未标定拒跑闸门：把“标没标过”变成可执行的检查（个人认为本线最有价值的一步）

问题的由来（实验二的教训）

实验二栽过一次：配置文件里几何如果填的是**编出来的占位值**，照样能通过项目里全部的结构自检——因为那些检查校验的是“三份配置之间一致不一致”，系统里**根本没有“是否标定过”这个概念**。在仿真里这无所谓（坐标错了，屏幕上看得见）；在真机上，那是一条真机械臂开到一个没人量过的位置。

做法

1. **未标定项一律写 null**——让“没标”在数据上就是一个**编不像**的值，而不是一个看着合理的数。占位项另外留 `_note` 标记，闸门会扫描这些标记做兜底。
2. **四份标定记录分开署名**（表 2）——这才知道**卡在谁那里**，而不是笼统地报“配置没填完”。
3. **闸门前置到“连机器人之前”**真机入口 `run_real.py` 启动的第一件事就是过闸门，未通过则直接退出，**相机不建、机器人不连**。
4. **不给“跳过闸门”的开关**：想跳过只能去改配置里的标定记录，而那是**留痕**的。

为什么做成硬门而不是警告

“未标定”这件事的风险不对称：误判“已标定”的代价是**真机撞桌或空抓**，而误判“未标定”的代价只是**多跑一次命令**。风险不对称时，默认拒绝比默认放行合理。

记录位置	负责人	内容
task.json → camera.calibration	组员乙	相机标定（标定模型 + 数值 + 反投影误差）
grid_cells.json → calibration	组长（现场量）	取物网格几何（卷尺量出每格两角坐标）
bins.json → calibration	组长	分类料盒中心
ep_waypoints.json → calibration	组长（本人）	8 个底盘位姿 + 臂两档 + 夹爪档位

表 2: 四份标定记录各自署名。任一份未填，真机入口拒绝启动。

3.5 失败即收尾：段号 → 原因 → 立刻回到已知状态

EP 是**开环**的：动作发出去之后，机械臂实际停在哪儿并没有可靠反馈。因此失败处置不能只“记一笔往下走”——那样下一个物体开始时就等于**拖着一条位置未知的臂横扫桌面**。本线的处理是三步（图 3）：

1. **段号定因**：失败发生在第几段，就决定了动作级原因是哪个（表 1），日志里能直接看出“是没夹起”还是“够不着”；
2. **立刻收尾 safe_reset**：开爪 → 抬到高点 → 回归位点，**尽力而为、不再抛错**（反正已经在失败路径上）；
3. **每次取放前先归位 prepare()**：上一次若是被人工中止或收尾也失败了，臂可能停在低处——所以每次动作前先抬到高位并开爪。这一步若失败**不吞**，直接停线。

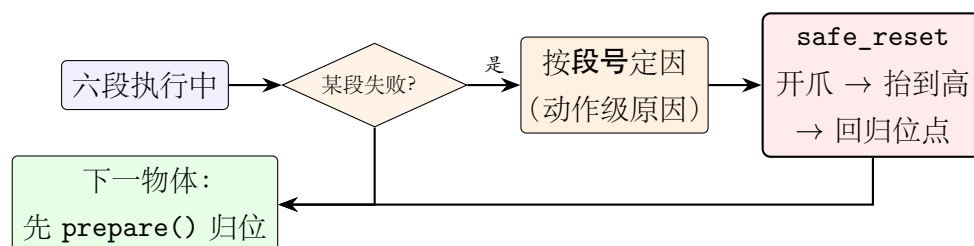


图 3: 失败路径：段号定因 → 立刻收尾 → 下一次动作前再归位。三级递进都是为了同一件事——**不让机械臂带着未知位姿继续工作**。

3.6 日志与验收口径：让“人工只做了该做的”也能被日志证明

真机的日志落在 `exp3_logs_real/run_<时间戳>/`，其中 `records.jsonl` 与 `result.json` 的 schema 与仿真线完全相同（可直接比对两线结果），另外多一份 `real_run.json` 记录真机特有的上下文：是否演练、四份标定记录、动作链步数、以及人工确认的次数。

最后一项是刻意留的凭据。EP 没有任何物体位姿感知，“夹住没有 / 放正没有”只能靠人确认；而 docx § 六要求“正常任务过程中不得进行人工类别判断和动作触发”。两者的边界是：

- **人不参与**：判类别（由识别模型给）、选目标与触发动作（由状态机给）；
- **人参与**：只回答“夹住了吗 / 放正了吗”这两个关于物理姿态的问题。

为了这条边界可核查，每条物体记录的 `detail` 字段会写明该次判断的来源——是“操作员确认夹住”还是演练口径的“未判假定成功”，二者在日志里用词不同、可区分；`real_run.json` 另记人工确认总次数（正常一轮 = $2 \times$ 物体数：每个物体问一次 HELD、一次 PLACED）。验收时“人到底参与了什么”可以由文件读出，而不是靠回忆。

4 项目中遇到的问题解决方法

问题 1：EP 的臂只有“一个水平自由度”，桌面横向覆盖从哪来。

定位：实验二的抓取动作可以直接照搬，但可达范围不能想当然。EP 工程形态的臂 `arm.moveto(x_mm, y_mm)` 中， x 是向前伸出的距离、 y 是高度——臂实际只在一个竖直平面内运动；官方行程约 0.22 m（水平） $\times$ 0.15 m（垂直），需要现场实测。

解决：把“横向覆盖”明确划给底盘：每个网格对应一个底盘停车位姿，车先开到位、再由臂在竖直平面内完成下探/抬升。于是执行侧需要标定的不是“一格一个关节角”，而是“一格一个底盘位姿 + 一组臂档位”。

个人思考：这条的教训是先量工作空间再定场景。如果按“桌面铺满网格、让臂去够”的思路先把网格间距定死，真机上会直接撞上行程墙；而后台的对策（底盘逐格跑）代价是标定点数与里程计漂移都翻倍。所以网格间距应当由“臂的实际可达域”倒推，而不是先画好网格再让机器人去凑。

问题 2：EP 没有物块感知，“抓没抓住”无法自动判——但验收又禁止人工干预。

定位：夹爪没有力/位反馈，相机在动作过程中也会被臂遮挡，所以“是否夹住”“是否放正”在硬件上无从自动判断。可 docx § 六明确要求正常过程中不得人工判类、不得人工触发动作。两者看起来是冲突的。

解决：把“人工干预”拆成两类区别对待——决策性的（判类别、选目标、发动作）全部由系统做；“物理状态确认”性的（夹住了吗、放正了吗）由人回答，且人的回答不会触发任何动作，只用于把这一物体记为成功或失败。并把这个边界写进日志 (§3.6)，使“人只做了确认”这件事可核查，而不是一句自述。

个人思考：“禁止人工干预”这类要求，真正要禁止的往往是人在回路里做判断、当传感器；而“人作为物理状态的确认者”与它是两件事。把这条边界显式写出来并留下日志证据，比在答辩时口头解释清楚得多。

问题 3：配置里“编的占位值”能骗过全部结构检查。

定位：项目原有的配置自检校验的是三份配置之间的一致性，没有“是否标定过”这个概念。于是随手编一组“看着合理”的坐标，自检会**全部通过**——在仿真里无所谓，在真机上就是让机械臂去一个没人量过的位置。

解决：见 §3.4 的四条：未标定项写 `null`（编不出像样的默认值）、占位项留标记、四份记录**分开署名**、**闸门前置到连机器人之前**且不给跳过开关。

个人思考：检查项“通过”不等于“前提成立”。一个只检查内部一致性的校验器，对“数据本身是否来自真实测量”是**盲**的；而这类盲区恰恰只在真机上才暴露。把关键前提变成**可执行的门**，比在流程文档里写一句“记得先标定”可靠。

问题 4：开环平台上的失败处置——失败之后机械臂停在哪儿没人知道。

定位：EP 的动作是开环的，失败时没有可靠的位姿反馈。若只记一笔就继续下一个物体，下一次动作就可能在“臂还停在桌面上方低处”的状态下开始，导致**拖着臂横扫**。

解决：三级防线 (§3.5)：失败即 `safe_reset`（开爪 → 抬到高 → 回归位点，尽力而为）；每次取放前 `prepare()` 先归位（失败则**拒绝动作**、宁可停线）；退回段自带一次重试——物已经放下了，不值得为一次读位抖动把整轮判失败，但连试两次都回不去就**真抛错停线**。

个人思考：在开环系统里，“**回到一个已知状态**”比“尽量把当前动作做完”重要得多。把收尾做成默认路径（而不是异常处理里的附加项），意味着每一段代码都必须回答“我失败之后，机器人处在什么状态、谁负责收拾”。

问题 5：SDK 的“动作完成”回调早于实际到位。

定位：EP 的原语调用返回**动作成功**的时刻，比位置推送更新的时刻**早约 0.5 s**（官方把结果放在成功位里，而“等它返回”并不等于“动作已完成”）。若按“命令返回了”就往下走，机械臂会在**移动途中**就开始抓取或搬运。

解决：不自己另写一份驱动，而是**直接复用实验二已经调好的那份 EP 驱动**（`ep/drive/rm.py`）。它内部的 `arm_moveto()` 已经带了“等位置推送追上、且连续两次读数一致才算稳定”的等待逻辑，本线的执行器调用它就自动继承了这套处理。

个人思考：这次复用是为了**避免两份驱动各自漂移**——同一份 SDK 的坑，抄第二遍一定会抄丢一部分。代价是公开快照里没有那份驱动，因此快照中只能做**不连硬件的演练**；这个取舍我认为是划算的。

问题 6：越界命令会被“安静地钳制”。

定位：当目标高度超出行程时，SDK 会把命令**钳制**到边界，臂“稳定地停在”一个并非目标的读数上。复用的驱动在检测到这种偏差时只打一条**警告日志**、**不抛错**——于是执行侧会返回“已到抓取位”，是一个**假成功**。

解决：运行时它仍会被兜住：钳制导致夹爪没对准物体 → 人工确认 `HELD` 为否 → 判 `grasp_failed` → 重抓一次 → 连续失败达上限 → 保护性停止。**行为是安全的**，但现象容易被误读成“抓取不稳定”。因此把这条**明确写进真机文档的已知限制**：现场若出现连续 `grasp_failed`，第一件事是去日志里找那条“未到位”警告，检查是不是标定值超了行程，而不是去怀疑夹爪。

个人思考：“不抛错但结果不对”是最难查的一类失败。对这种情形，即便代码选择继续运行，也必须在文档里留下路标，否则下一次联调还要重新推一遍。

附：本人这条线的复现方式

```
# 1) 离线全套单测（无需 ROS、无需真机、无需相机）
cd exp3 && python3 -m unittest discover -s tests -t . -v # 119 tests OK

# 2) 真机标定：读数直接写回位姿表，不用手抄（步骤见 real/标定说明.md）
python3 real/calibrate.py --odom # 推着车到位，实时看 odom
python3 real/calibrate.py --record c1 # 记该格底盘位姿
python3 real/calibrate.py --record-arm grab 74 120 # 试臂档（读回差 >3mm 拒绝入
    表）
python3 real/calibrate.py --seal --by <名字> --venue <场地>

# 3) 标定进度自查（未标定项逐个点名，含负责人）
python3 config_check.py --config-dir config_real
python3 config_check.py --config-dir config_real --require-calibrated # 闸门口
    径，必须 0 项

# 4) 演练：不连 EP、不动真机，把整条动作链打出来（标定填完后先跑这个）
python3 real/run_real.py --config-dir config_real --dry-run \
    --camera none --detector mock --assume-judge --no-pause

# 5) 真机首跑：每个物体动手前等一次回车（人盯；本轮记为 rehearsal）
python3 real/run_real.py --config-dir config_real --confirm-each \
    --camera cv2:0 --detector file --detector-file <检测器文件>

# 6) 真机记分轮：三个演练开关全关，HELD / PLACED 向操作员确认
python3 real/run_real.py --config-dir config_real \
    --camera cv2:0 --detector file --detector-file <检测器文件>
# 日志：exp3_logs_real/run_<时间戳>/{run.log, records.jsonl, result.json,
    real_run.json}
# real_run.json 记 evidence_grade 与 outcome，验收时据此判这份结果能不能用
```

说明：上列第 1-3 项（119 个离线用例，含逐段失败注入、闸门拦截、标定工具的钳制读数拦截、四类异常的端到端复跑）在当前仓库中均可直接执行并已有结果。第 4-6 项中涉及 EP 硬件的部分属阶段二待完成——本报告所述真机侧的设计与失败处置，凡未标注“现场实测”者，均为离线（假机械臂）验证的结论，真机实测结果以现场记录与验收日志为准。相关代码已整理在 exp3/real/（执行后端、标定工具、入口、相机与检测器）与 exp3/config_real/（真机配置与

位姿表) 中。